物联网系统设计

音频识别与健康监测

姓名: 消瘦的猪猪

日期: 2025-4-19

Contents

1 项目介绍 **3**

2 模块介绍 **4**

2.1 音频接收电路 4

2.1.1 MAX9814 模块 4

2.1.2 直流偏置电路 5

2.2 ADC 采样、FFT 算法、类型判断 6

2.2.1 ADC 采样 6

2.2.2 FFT 算法 6

2.2.3 类型判断 8

2.3 体温检测 10

2.4 心率血氧检测 11

2.5 风扇控制 13

2.6 屏幕显示 14

2.7 天问语音模块 15

3 华为云开发 **16**

4 微信小程序开发 **16**

4.1 小程序开发 16

4.2 js 文件 18

4.3 界面设置 19

5 结果展示 **22**

6 项目总结 **22**

1 项目介绍

- 本项目设想的使用场景是通过微信小程序远程获取产品所在区域的音频特性，如果是人声，可以判断对方的性别和情绪，如果是音乐声，可以获取音乐的主频和频谱质心。同时通过实时体温检测、心率检测、血氧检测，可以进一步判断对方的情绪以及健康状况，本项目通过 MAX9814 芯片处理麦克风接收到的音频信号，并设置 50dB 增益，放大后的信号存在负电压，而单片机 ADC 并不能读取负电压，所以使用 LM358 双运放芯片设计了一个直流偏置电路，经过直流偏置电路后的电信号，传输给单片机 ADC 采样，单片机对采样的 1024 个点进行快速傅里叶变换（FFT），获取主频、谐噪比、谐波比、频谱质心、基频范围等数据，通过这些数据判断是音乐还是人声。本项目还有一个附加功能，就是通过微信小程序控制风扇转动，也可以通过语音控制风扇转动，从而起到了调整人体温度的作用。

- 项目整体框图如下

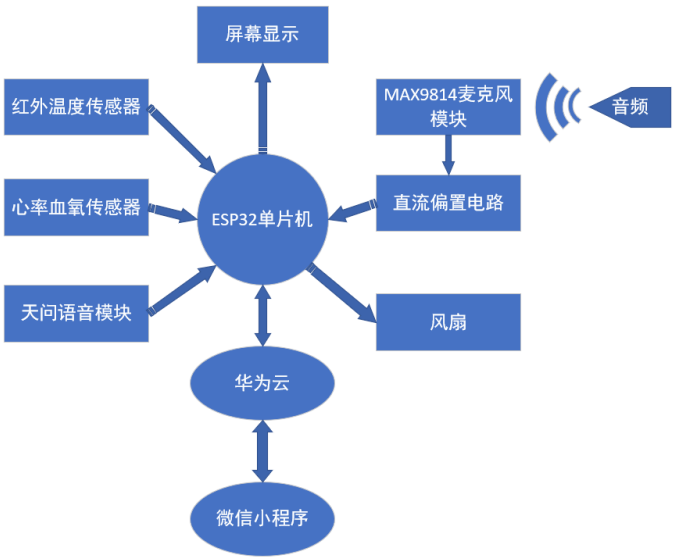


Figure 1: 项目框图

- 项目实物图如下

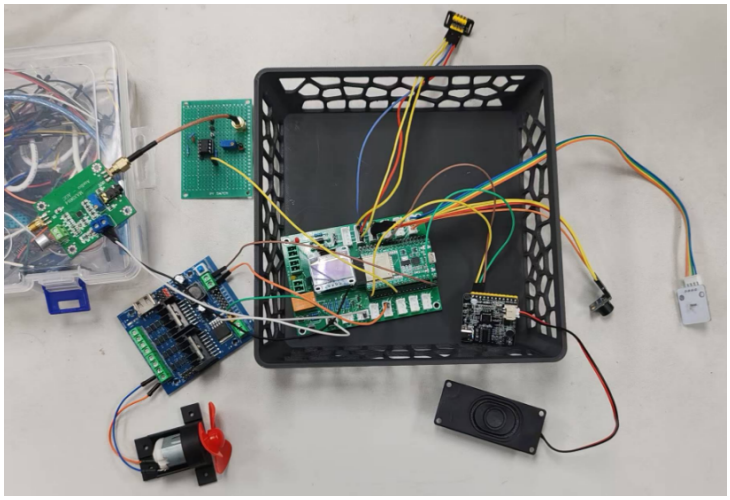


Figure 2: Caption

2 模块介绍

2.1 音频接收电路

2.1.1 MAX9814 模块

- MAX9814 模块有 40dB、50dB、60dB 三种增益选择，具有简易滤波电路，可以滤去白噪声，使用该模块进行音频接收

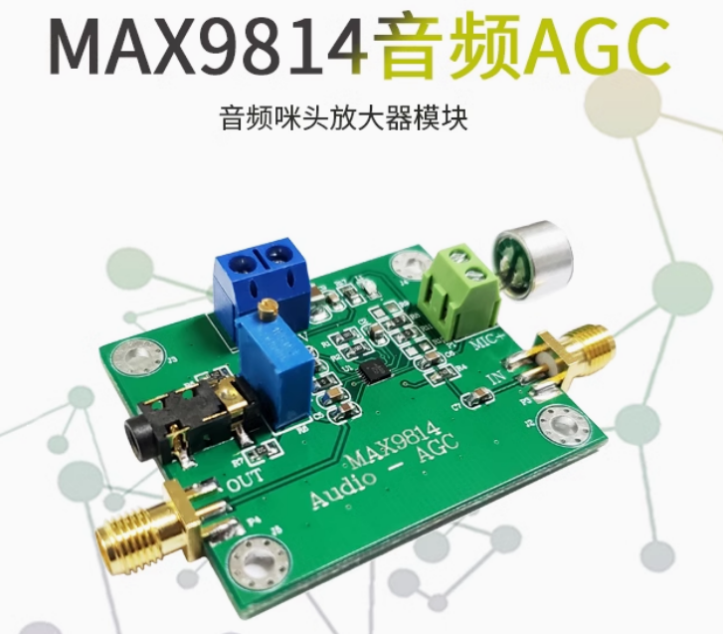


Figure 3: MAX9814

2.1.2 直流偏置电路

- 使用 LM358 双运放芯片设计的直流偏置电路如下：

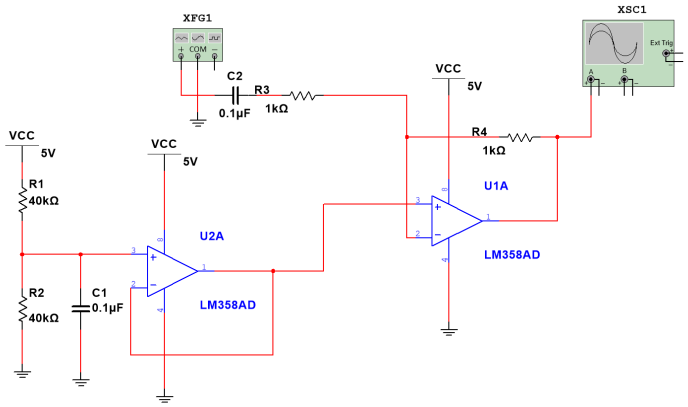


Figure 4: 直流偏置电路仿真

- 绘制的 PCB 板图如下：（与仿真电路不同的是，为了输入信号增益可调，使用电位器替换了一个定值电阻。）

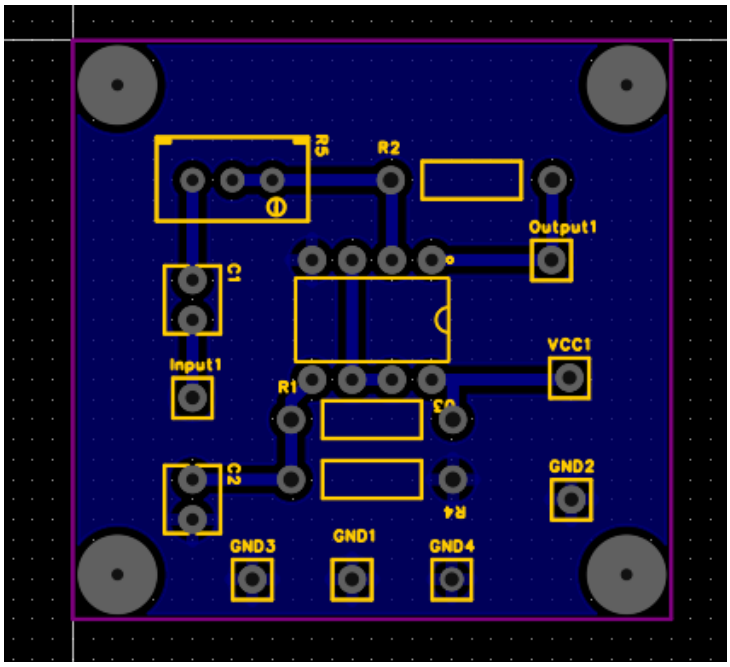


Figure 5: 直流偏置电路 PCB

- 但在制作出 PCB 后发现同轴线底座不能连接杜邦线，所以实际电路制作使用的是洞洞板。

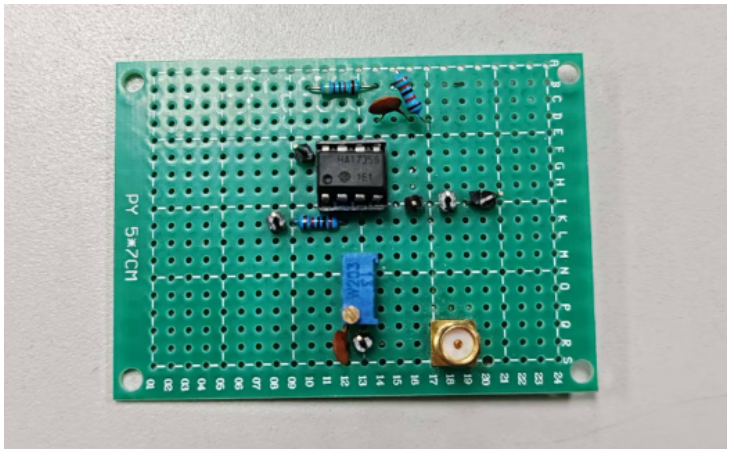


Figure 6: 实际电路

2.2 ADC 采样、FFT 算法、类型判断

2.2.1 ADC 采样

- ADC 采样使用的是管脚 33。
- 采样点数设置为 1024，考虑到人说话的声音普遍频率比较低，一般低于 1kHz，音乐频率较高，但主频一般不会高于 5kHz，根据奈奎斯特采用准则，设置采样频率为 10kHz 即可满足要求。

2.2.2 FFT 算法

- FFT 初始化与配置：（加入了汉宁窗，避免漏频）

```
ArduinoFFT<double> FFT;
double vReal[SAMPLES];
double vImag[SAMPLES];
double windowingFactors[SAMPLES];

FFT.windowing(vReal, SAMPLES, FFT_WIN_TYP_HANN,
    ↪ FFT_FORWARD, windowingFactors);
```

- 音频采样与预处理：

```
unsigned long samplingStart = micros();
for(int i=0; i<SAMPLES; i++) {
    vReal[i] = analogRead(ADC_PIN);
    delayMicroseconds(static_cast<int>(1000000.0f /
        ↪ SAMPLING_RATE) - (micros() - sampleStart));
}
```

```

float dcOffset = sum / SAMPLES;
for(int i=0; i<SAMPLES; i++) {
    vReal[i] = (vReal[i] - dcOffset) * (VREF / ADC_MAX);
    vImag[i] = 0;
}

FFT.windowing(vReal, SAMPLES, FFT_WIN_TYP_HANN, FFT_FORWARD, windowingFactors);
```

- 执行 FFT 计算:

```
FFT.compute(vReal, vImag, SAMPLES, FFT_FORWARD);
FFT.complexToMagnitude(vReal, vImag, SAMPLES);
```

- 特征提取关键函数

```
  double peakFreq = FFT.majorPeak(vReal, SAMPLES, SAMPLING_RATE);
  float totalEnergy = 0, spectralCentroid = 0, entropy = 0, spectralFlux = 0;

  for (int i = 1; i < SAMPLES / 2; i++) {
    float freq = i * (SAMPLING_RATE / float(SAMPLES));
    float mag = vReal[i];
    totalEnergy += mag;
    spectralCentroid += freq * mag;

    if (totalEnergy > 0) {
      float prob = mag / totalEnergy;
      if (prob > 0) entropy -= prob * log(prob);
    }

    spectralFlux += pow(mag - prevMagnitudes[i], 2);
    prevMagnitudes[i] = mag;
  }
  spectralCentroid /= totalEnergy;
  spectralFlux = sqrt(spectralFlux);
```

2.2.3 类型判断

- 判断音频类型（即判断是人声还是音乐声）用到了classifyAudio()函数，判断的依据结合了基频范围、频谱质心和谐噪比

```
bool classifyAudio(float peakFreq, float spectralCentroid, float entropy,float spectralFlux, float hnr, float energyVariation,float totalEnergy) {
bool inVocalRange = (peakFreq > 70 && peakFreq < 1200);
bool centroidCheck = spectralCentroid < 4500;
bool entropyCheck = entropy < 22.0;
bool hnrCheck = hnr > 0.15;
bool energyCheck = totalEnergy > 1.0;

bool hasHarmonics = checkHarmonicStructure(vReal, SAMPLES, peakFreq, SAMPLING_RATE);

bool isLikelyMusic = false;
if (spectralCentroid > 4000 && entropy > 20) {
isLikelyMusic = true;
}
if (spectralFlux > 4.0) {
isLikelyMusic = true;
}
if (peakFreq < 40 || peakFreq > 2000) {
isLikelyMusic = true;
}

bool isSpeech = false;
if (!isLikelyMusic) {
isSpeech = inVocalRange ||
(centroidCheck && entropyCheck) ||
hasHarmonics ||
hnrCheck;
}

if (isSpeech) {
speechState.lastWasSpeech = true;
speechState.lastSpeechTime = millis();
speechState.lastPeakFreq = peakFreq;
```

```
}
else if (isLikelyMusic) {
speechState.lastWasSpeech = false;
}
else {
isSpeech = speechState.lastWasSpeech;
}

return isSpeech;
}
```

- 判断情绪用到了`calculateEnergyVariation()`函数和`extractFFTFeatures()`函数，前者是用来计算能量波动的，后者是用来提供频带能量分布的，如果一个人处于情绪波动比较大的状态，音频的能量波动会比较大，而且高频带能量占比会比较高。人的情绪是多样的，有开心、愤怒、平静、怀疑、惊讶，区分起来有一定的难度，需要调整很多参数，而且判断的结果可能存在偏差，所以为了提升判断的准确度，本项目仅区分 angry 和 normal 两种情绪。人的情绪总是具有连续性的，即人不会在 angry 和 normal 之间反复变化，为了提升系统的鲁棒性，设置了状态计数器`emotionCounter`，最大值为 3，最小值为-2，如果判断情绪为 angry，则计数器加 1，反之，计数器减 1。

```
if (avgLoudness >  noiseFloor) {
  if(emotionCounter <= 3) emotionCounter =
     ↪ emotionCounter + 2;
  if(emotionCounter > 0){
    emotion = "Angry";
    emotionresult = 0;
    analogWrite(RED, 0);
    ColorRed = 255;
    analogWrite(GREEN, 255);
    ColorGreen = 0;
  }
} else {
  if(emotionCounter >= -1) emotionCounter =
     ↪ emotionCounter - 1;
  if(emotionCounter <= 0){
    emotion = "Normal";
    emotionresult = 1;
    analogWrite(RED, 255);
    ColorRed = 0;
```

```
    analogWrite(GREEN, 0);
    ColorGreen = 255;
  }
}
```

- 男女性别判定主要是基于男性主频比较低、频谱质心比较低，女性主频比较高、频谱质心比较高。这只是普遍规律，存在音调比较高的男性，也存在声音低沉的女性，而且孩童的声音频谱与女性极为近似，所以性别判断有时会出错。同样，避免频繁切换结果，设置了genderCounter性别计数器用于防抖。

```
if (peakFreq < 500 && spectralCentroid < 3000) {
    if (genderCounter <= 0) {
        gender = "Male";
        genderresult = 0;
    }
    genderCounter--;
} else {
    if (genderCounter > 0) {
        gender = "Female";
        genderresult = 1;
    }
    genderCounter++;
}
```

2.3 体温检测

- 体温检测需要调用Adafruit_MLX90614库
- 使用mlx.begin()初始化传感器，即可开始体温检测

Figure 7: 红外测温模块

2.4 心率血氧检测

- 传感器初始化

```
#include "MAX30105.h"
#include "heartRate.h"
#include "spo2_algorithm.h"

MAX30105 particleSensor;

void setup() {
//MAX30105 Init
    if (!particleSensor.begin(Wire, I2C_SPEED_FAST)) {
        Serial.println("MAX30105未找到, 请检查连线! ");
        while (1);
    }

    byte ledBrightness = 60;
    byte sampleAverage = 4;
    byte ledMode = 2;
    byte sampleRate = 100;
    int pulseWidth = 411;
    int adcRange = 4096;
```

```
    particleSensor.setup(ledBrightness, sampleAverage, ledMode, sampleRate, pulseWidth, adcRange);
    particleSensor.setPulseAmplitudeRed(0x0A);
    particleSensor.setPulseAmplitudeIR(0x0A);
}
```

- 心率与血氧计算

```
#define BUFFER_SIZE 100
uint32_t irBuffer[BUFFER_SIZE];
uint32_t redBuffer[BUFFER_SIZE];

int32_t heartRate, spo2;
int8_t validHeartRate, validSPO2;

void loop() {
    for (int i = 0; i < BUFFER_SIZE; i++) {
        while (!particleSensor.available()) {
            particleSensor.check();
        }
        redBuffer[i] = particleSensor.getRed();
        irBuffer[i] = particleSensor.getIR();
        particleSensor.nextSample();
        delay(10);
    }

    maxim_heart_rate_and_oxygen_saturation(
        irBuffer, BUFFER_SIZE, redBuffer,
        &spo2, &validSPO2,
        &heartRate, &validHeartRate
    );

    if (validHeartRate && validSPO2) {
        Serial.print("心率: "); Serial.print(heartRate); Serial.print(" bpm");
        Serial.print("\t血氧: "); Serial.print(spo2); Serial.println("%");
    } else {
        Serial.println("测量无效, 请确保手指稳定覆盖传感器! ");
    }
```

```

    for (int i = 25; i < BUFFER_SIZE; i++) {
        redBuffer[i-25] = redBuffer[i];
        irBuffer[i-25] = irBuffer[i];
    }
}
```

- MAX30105 模块实物图如下：

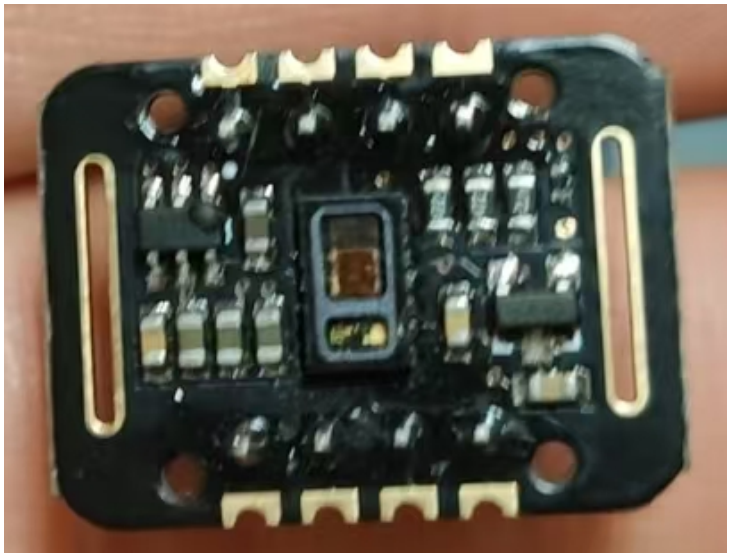


Figure 8: 心率血氧检测模块

2.5 风扇控制

- 风扇是通过一个小型有刷电机控制的，5V 驱动电机。在用单片机的 5V 的 IO 口直接控制电机时发现，电机不够稳定而且停转瞬间特别快，会产生较大的反电动势，电流过大，有时会看到明显的烟，而且单片机上的二极管摸起来明显发烫。为了避免电机损坏单片机，我通过单片机控制电机驱动模块（L298N）间接控制风扇转动。控制的逻辑是使用微信小程序或者天问语音模块控制单片机，将`Switch`的值设置为 1，即闭合开关，风扇转动；将`Switch`的值设置为 0，即打开开关，风扇停转。

Figure 9: 风扇实物图

- 5 到 7V 为 L298N 的逻辑高电平，可以通过单片机输入具有确定占空比的 PWM 信号控制 L298N 输出较大功率的方波信号。一共有四路输出，选择一路即可。

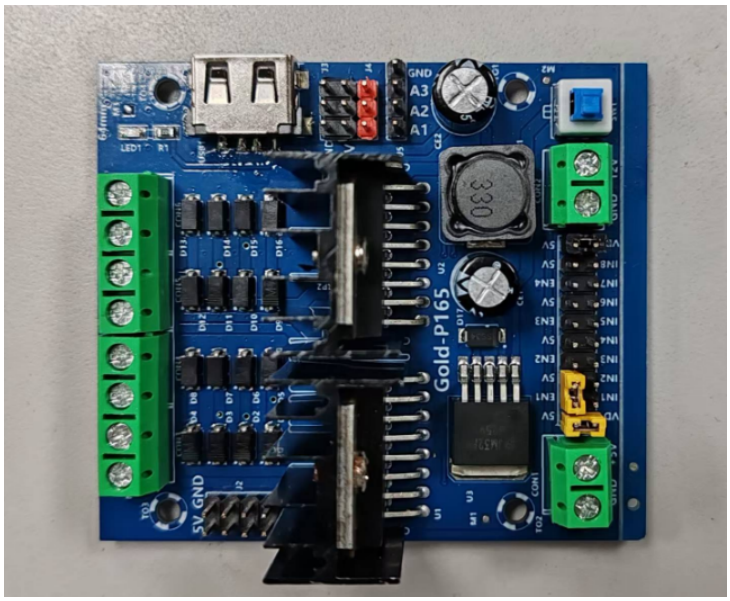


Figure 10: L298N

2.6 屏幕显示

- 屏幕是用来显示实时监测到的音频类型和频谱
- 如果判断是人说话声，就显示`Speech`，并显示性别和情绪；如果判断是音乐声，就显示`Music`，并显示主频和质心。具体代码实现如下

```
display.clearDisplay();
display.setCursor(0, 0);
display.print(isSpeech ? "SPEECH" : "MUSIC");

if (isSpeech) {
```

```
display.setCursor(0, 16);
display.print("Gender: "); display.print(gender);
display.setCursor(0, 32);
display.print("Emotion: "); display.print(emotion);
} else {
display.setCursor(0, 16);
display.print("Freq: "); display.print(peakFreq, 1);
display.print("Hz");
display.setCursor(0, 32);
display.print("Cent: "); display.print(spectralCentroid,
0); display.print("Hz");
}
display.display();
}
```

2.7 天问语音模块

- 天问语音模块配置如下

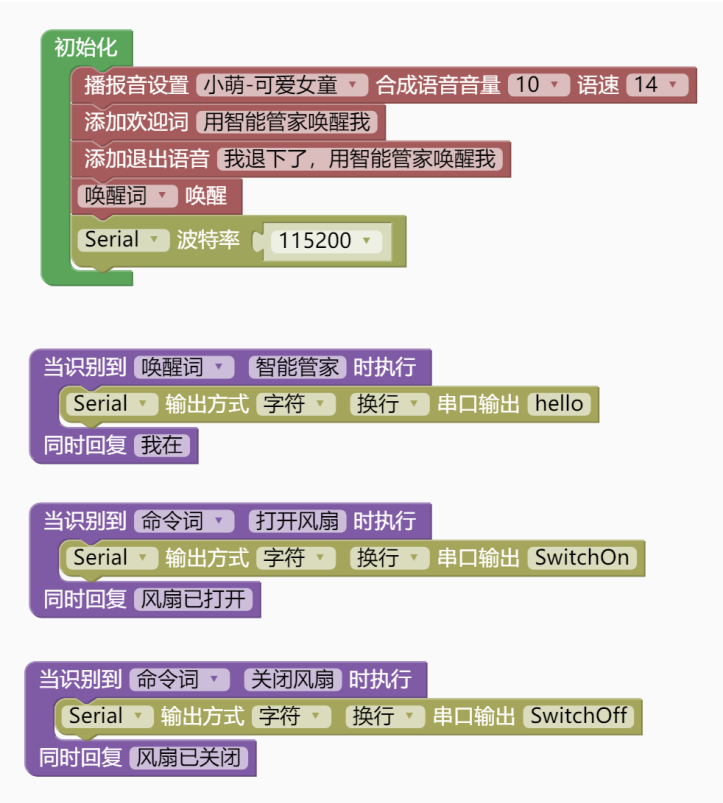


Figure 11: 天问语音模块配置

- 单片机接收语音模块传来的命令，并根据命令修改Switch的值，从而控制风扇转

动

```
if (mySerial.available()>0){
  delay(10);
  tianwen=mySerial.readString();
  if (tianwen!="" && tianwen.length()>2){
    const char * tianwentemp=tianwen.c_str();
    Serial.println(tianwen);

    if(strstr(tianwentemp,"SwitchOn")!=
       ↪ NULL){Switch=1;}
    if(strstr(tianwentemp,"SwitchOff")!=
       ↪ NULL){Switch=0;}
   }
}
```

3 华为云开发

- 添加了十一种属性，分别是Temperature（人体体温）、RED（红灯亮度）、GREEN（绿灯亮度）、Time（时间）、Frequency（主频）、Emotion（情绪）、Switch（风扇开关）、Gender（性别）、isSpeech（是否是人声的判断）、spo2（血氧）、heartRate（心率）
- 添加了一个命令Switch，用来配合微信小程序控制风扇打开和关闭

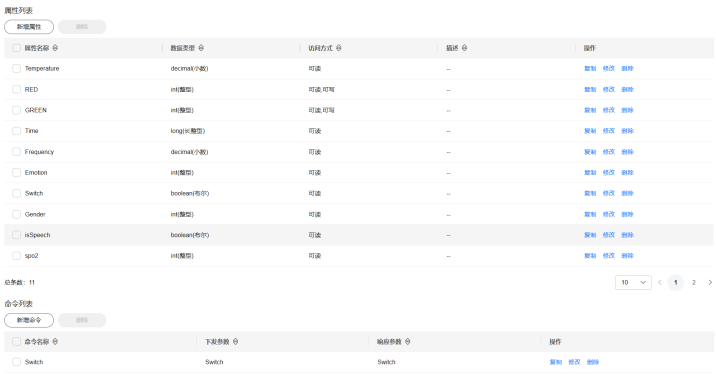


Figure 12: 新增属性和命令

4 微信小程序开发

4.1 小程序开发

- 获取 AppID

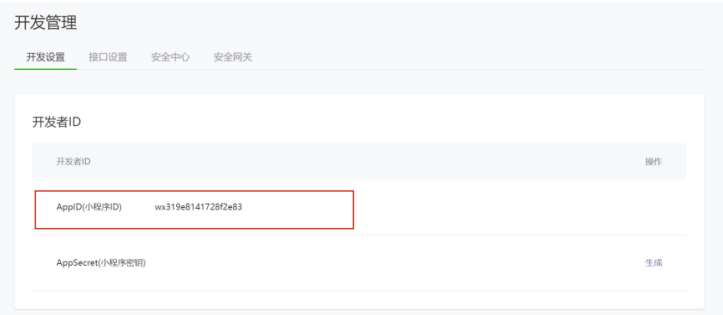


Figure 13: 获取 AppID

- 创建项目
- 设置用户登录参数，`projectId`和`deviceId`分别是项目 ID 和设备 ID

Figure 14: 用户登录参数

- projectId 的获取：点击右上角账户名，在列表中选择我的凭证

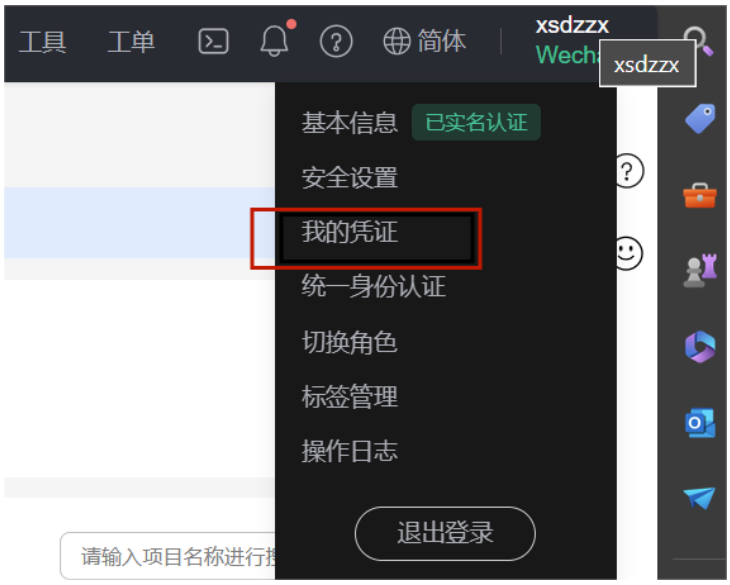
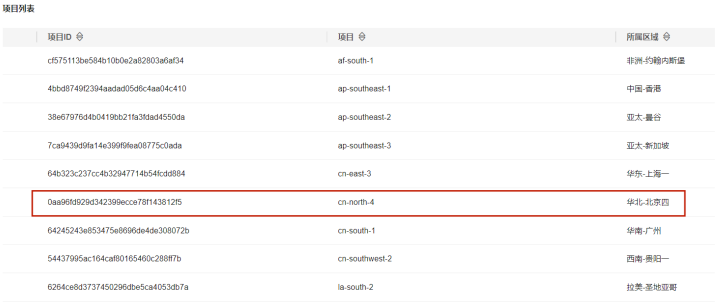


Figure 15: 我的凭证

- 在项目列表中复制对应区域的 projectId，我使用的是华北-北京四的服务器



项目列表

项目ID	项目	所属区域
cf575113be584b10b0e2a82803a6af34	af-south-1	非洲-约翰内斯堡
4bbd8749f2394aadad05d6c4aa04c410	ap-southeast-1	中国-香港
38e679764b04190b21fa3fdad4550da	ap-southeast-2	亚太-曼谷
7ca9439d9fa14e399f9fea08775c0ada	ap-southeast-3	亚太-新加坡
64b323c237cc4b32947714b54fcdd884	cn-east-3	华东-上海一
0aa96fd929d342399ecce78f143812f5	cn-north-4	华北-北京四
64245243e853475e8696de4de308072b	cn-south-1	华南-广州
54437995ac164caf80165460c288ff7b	cn-southwest-2	西南-贵阳一
6264ce8d3737450296dbe5ca4053db7a	la-south-2	拉美-圣地亚哥

Figure 16: 获取 projectId

- 在服务器域名的服务器配置一栏，添加三个域名

服务器域名

本月还可修改42次 修改

使用 微信云开发 或 微信云托管，无需配置服务器域名，省心省力。如业务访问需要安全防护，可使用 安全网关，防爬防刷防攻击，实现业务安全。点击了解更多 域名配置要求

服务器配置	域名	可配置数量
request合法域名	https://c4e74a6b9c.st1.iotda-app.cn-north-4.myhuaweicloud.com https://iam.cn-north-4.myhuaweicloud.com https://iotda.cn-north-4.myhuaweicloud.com	300个

Figure 17: 添加域名

4.2 js 文件

- js 文件中主要代码部分为获取 token 和获取设备影子

```
gettoken: function () {
  console.log("开始获取token。。。");
  var that = this;
  wx.request({
    url: "https://iam.cn-north-4.myhuaweicloud.com/v3/auth/tokens",
    data: '{"auth": { "identity": {"methods": ["password"],"password": {"user": {"name": "${username}","password": "${password}","domain": {"name": "${domainname}"}}}},"scope": {"project": {"name": "cn-north-4"}}}}',
    method: "POST",
    header: { 'content-type': 'application/json' },
    success: function (res) {
      console.log("获取token成功");
      var token = JSON.stringify(res.header['X-Subject-Token']).replaceAll("\"", "");
      console.log("获取token=\n" + token);
      wx.setStorageSync('token', token);
      that.startPolling(); // 获取token成功后开始轮询
    },
    fail: function () {
      console.log("获取token失败");
    },
    complete: function () {
      console.log("获取token完成");
    }
  });
},
```

Figure 18: 获取 token

```
getshadow: function () {
  console.log("开始获取影子");
  var that = this;
  var token = wx.getStorageSync('token');
  wx.request({
    url: `https://c4e74a6b9c.st1.iotda-app.cn-north-4.myhuaweicloud.com/v5/iot/${projectId}/devices/${deviceId}/shadow`,
    method: 'GET',
    header: { 'content-type': 'application/json', 'X-Auth-Token': token },
    success: function (res) {
      console.log(res);
      var shadow = JSON.stringify(res.data.shadow[0].reported.properties);
      console.log('设备影子数据: ' + shadow);

      Temp = parseFloat(res.data.shadow[0].reported.properties.Temperature).toFixed(2);
      Freq = JSON.stringify(res.data.shadow[0].reported.properties.Frequency);
      var emotionValue = res.data.shadow[0]?.reported?.properties?.Emotion ?? 0;
      Emotion = emotionValue === 1 ? 'Normal' : 'Angry';
      var genderValue = res.data.shadow[0].reported.properties.Gender;
      Gender = genderValue === 1 ? 'Female' : 'Male';
      Spo2 = JSON.stringify(res.data.shadow[0].reported.properties.spo2);
      HeartRate = JSON.stringify(res.data.shadow[0].reported.properties.heartRate);
      var isSpeechValue = res.data.shadow[0].reported.properties.isSpeech;
      isSpeech = isSpeechValue === 1 ? 'Music' : 'Speech';

      console.log('温度=' + Temp + '℃');
      console.log('频率=' + Freq + 'Hz');
      console.log('情绪=' + Emotion);
      console.log('性别=' + Gender);
      console.log('血氧=' + Spo2 + '%');
      console.log('心率=' + HeartRate + 'bpm');
      console.log('音频类型=' + isSpeech);
      // 更新页面数据
      that.setData({
        result: `温度:${Temp}℃ 频率:${Freq}Hz 情绪:${Emotion} 性别:${Gender} 血氧:${Spo2}% 心率:${HeartRate}bpm 音频:${isSpeech}`,
        temperature: Temp,
        frequency: Freq,
        emotion: Emotion,
        gender: Gender,
        spo2: Spo2,
        heartRate: HeartRate,
        isSpeech: isSpeech
      });
    },
```

Figure 19: 获取设备影子

4.3 界面设置

- 设计了两个界面，界面 1 用来实时显示温度、血氧、心率，并控制风扇转动，界面 2 用来显示音频相关的内容，如果是人声就显示人的性别和情绪，如果是音乐声就显示音乐的主频。
- 界面采用卡片化设计，即插入图片使得微信小程序上每个界面的功能看起来更加直观，而且界面看起来更美观。

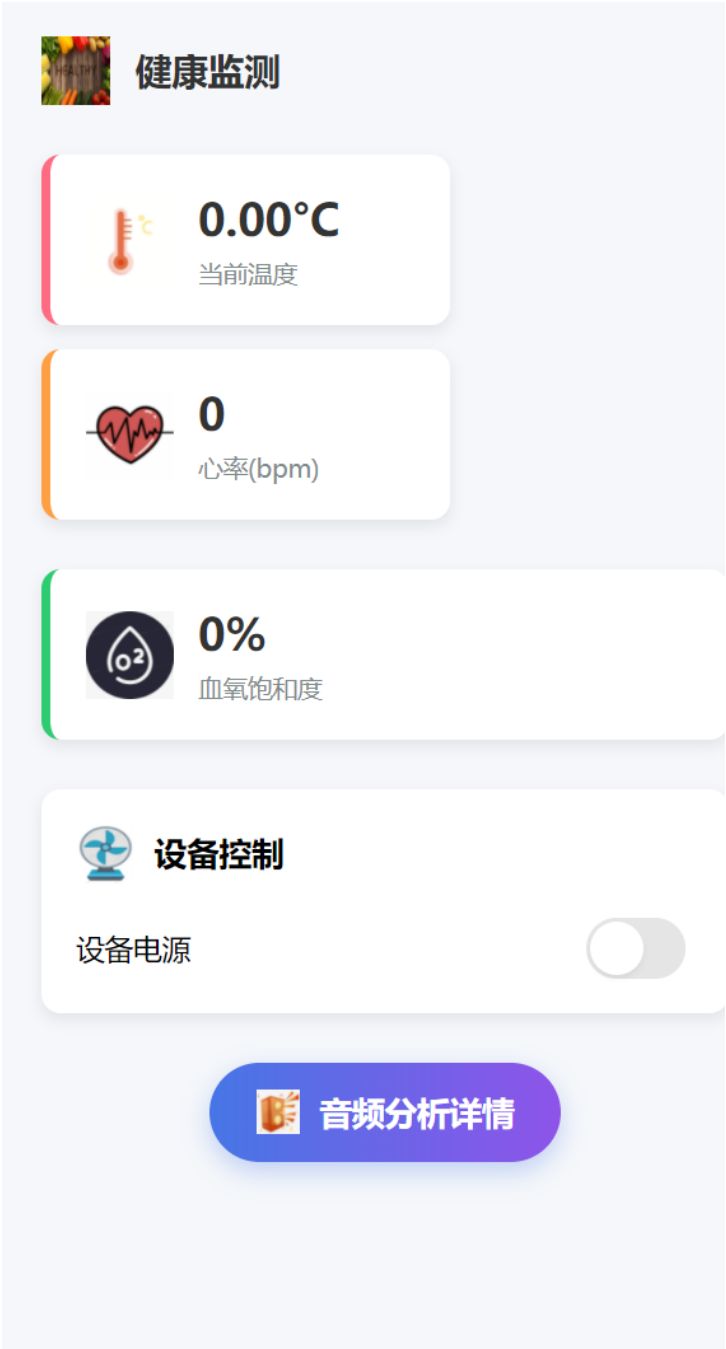


Figure 20: 健康检测界面

Figure 21: 音频检测界面

- `wxss`文件和`wxml`文件具体代码见附件 1

5 结果展示

- 展示视频分为七个部分，分别为
  - 音频接收电路检测
  - 情绪检测
  - FFT 算法检测
  - 音乐检测
  - 温度、心率、血氧检测
  - 微信小程序控制风扇转动
  - 天问语音模块控制风扇转动
- 演示视频见附件 2，软件源代码见附件 3

6 项目总结

最后制作出的产品可以轮循采样并进行频域分析，可以实现人声音乐声的判断、性别判断、情绪判断，能够通过外接传感器检测人体体温，将拇指放在心率血氧传感器上，可以实时检测人体血氧和心率，能够通过语音控制风扇的开闭，也可以通过微信小程序控制风扇的开闭。

单片机主程序代码行数总计 843 行，音频检测判断相关部分代码占比较高。在整个项目开发过程中，遇到的最大的困难是调参。想要根据人说话声尽可能准确地判断人的性别和情绪，不仅需要获取采集到的音频的主频，还需要通过相应的算法计算得到频谱质心、谐波比等，但是仅获取这些数据是远远不够的，最重要的是能够根据得到的数据判断到底是人声还是音乐声，是男生还是女生，是愤怒还是正常。所有的这些判断都需要划定一个范围，比如说主频大于 1kHz 就更有可能是音乐声，划定范围需要花费大量的时间去调试。

关于语音控制功能，我刚开始设想的是通过对频域的分析来控制风扇的开闭，先录入 5 段音频，比如说“开灯”两个字录入 5 次，在训练完成后，当我说出”开灯“两个字后，灯光亮起。为此我调用常用的小型音频模型 KNN，使用封装好的函数来存储和对比音频。但实际操作的难度是比较大的，一方面是人每次说话的频谱是不一样的，如果提前录好“开灯”，每次都使用这个录制好的音频，虽然可以更好地匹配存储的频谱，但是这会影响产品的使用，所以并不是一个好的选择。另一方面，我是通过串口来控制时候开始录入音频，在我通过串口发送命令后，就已经开始录入了，但是只有 1024 个采样点，10kHz 的采样频率，这就意味着采样的时间段是很短的，近似为 0.1s，可能还没有开始说“开灯”两个字，采样就已经结束了。我认为想要只通过频域分析来识别语音难度较大，可能还需要部署性能更强的音频模型或者添加机器学习等，所以在经过两

天的尝试后，我决定直接使用天问语音模块，语音模块与单片机通信，从而控制风扇的开闭。

这次项目的设计和开发过程让我对物联网有了更深刻的认识，提升了我在单片机程序设计、云服务器开发、微信小程序开发等方面的能力，我也使用到了很多新的模块和传感器，这对我以后进行其他嵌入式系统的开发有很大的帮助。未来，我希望能够进一步探索人工智能在物联网中的应用，例如利用深度学习优化音频分类和情绪识别的准确性，或者结合边缘计算技术，减少对云服务器的依赖，提高系统的实时性和可靠性。总的来说，这次项目不仅是一次技术实践，更是一次综合能力的提升。它让我更加明确了未来的学习方向，也增强了我在嵌入式系统和物联网领域继续深耕的信心。